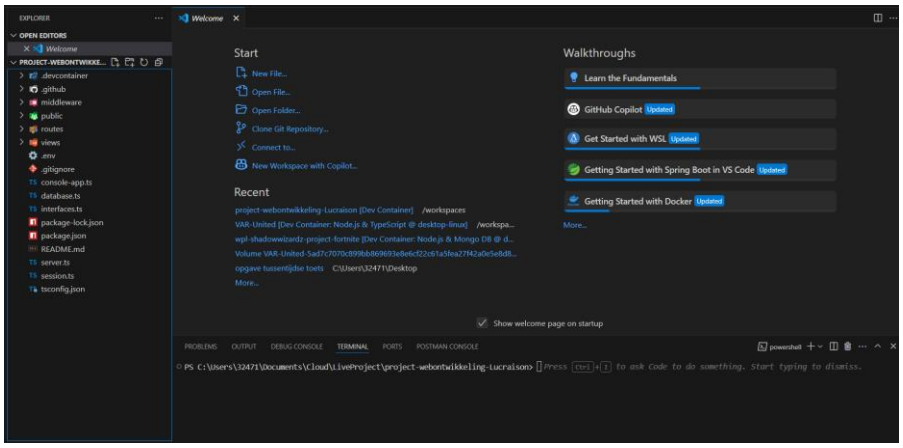
Stappenplan: Webapplicatie Live Zetten op Azure

Nicolas Herrera Santibañez

1PRO_D2

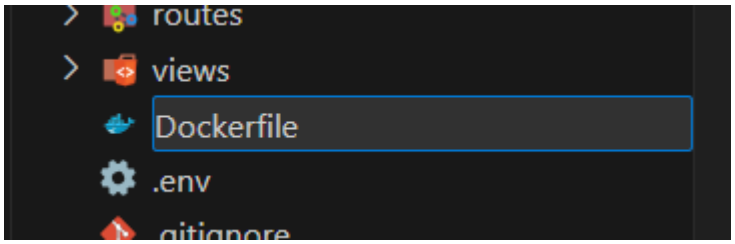
Stap 1:

Zet jezelf in de root van het project dat je wilt live zetten, in dit geval besloot ik voor mijn project van webontwikkeling



Stap 2:

We maken een nieuwe bestand aan “Dockerfile”, deze zal ons helpen om een Docker-container te bouwen.



Stap 3:

Dus we gebruiken een lichte Node.js 18 image als basis, werken in de directory /app, kopieren de package.json en eventueel package-lock.json naar de container, installeren alle dependencies en kopieren naar de container. Dan gewoon runnen.

```
Dockerfile > ...
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

RUN npm run build

CMD ["npm", "start"]
```

Stap 4:

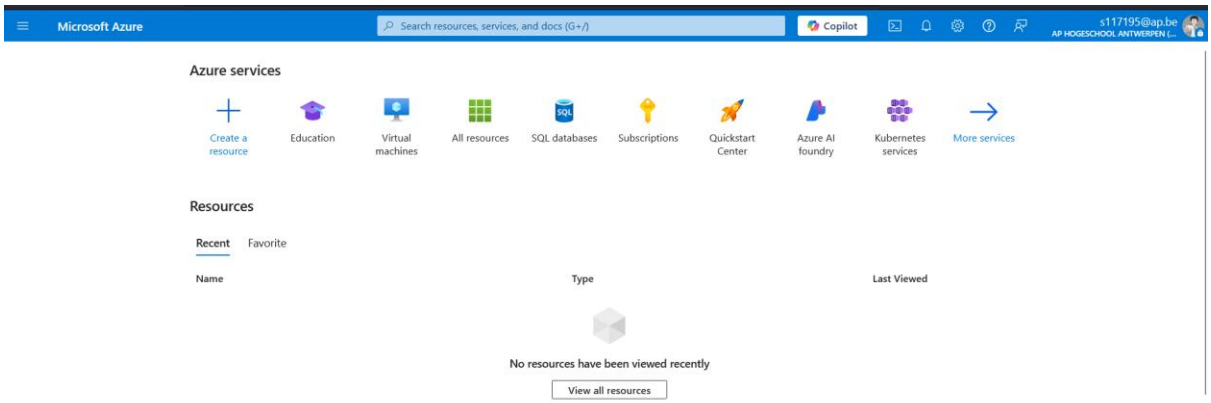
Met **docker build -t artists-site .** bouw je een Docker image op basis van de Dockerfile.

De vlag **-t** geeft de image de naam artists-site, en de **.** betekent dat het huidige pad gebruikt wordt als context.

```
PS C:\Users\32471\Documents\Cloud\LiveProject\project-webontwikkeling-Lucraison> docker build -t artists-site .
 [+] Building 9.2s (4/10)
  => [internal] load build definition from Dockerfile
  => => transferring dockerfile: 175B
  => [internal] load metadata for docker.io/library/node:18-alpine
  => [internal] load .dockerignore
  => => transferring context: 2B
  => [1/6] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
  => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
  => => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 11.53MB / 40.01MB
  => => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB
  => => sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 7.67kB / 7.67kB
  => => sha256:929b04d7c782f04f615cf785488fed452b6569f87c73ff666ad553a7554f0006 1.72kB / 1.72kB
  => => sha256:ee77c6cd7c1886ecc802ad6cedef3a8ec1ea27d1fb96162bf03dd3710839b8da 6.18kB / 6.18kB
  => => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
  => => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B
  => => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870
  => [internal] load build context
  => => transferring context: 11.83MB
```

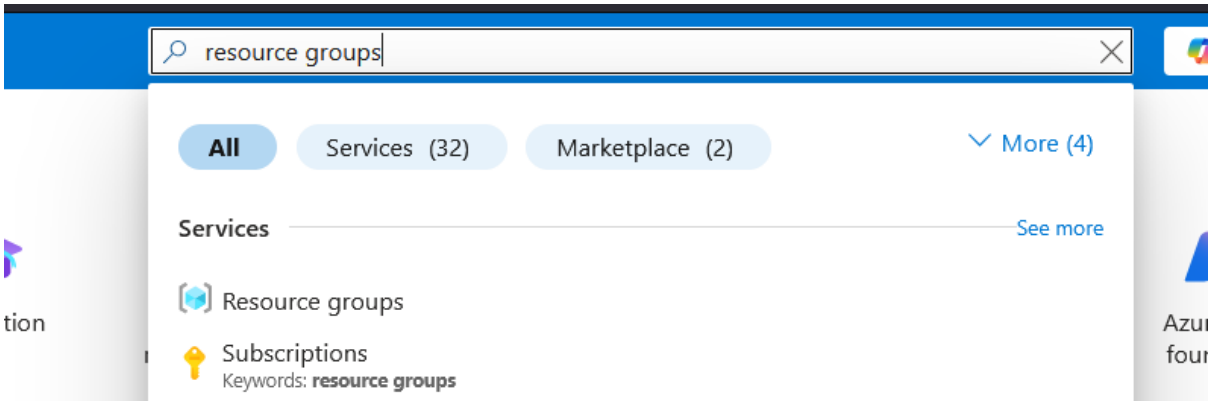
Stap 5:

Ga naar het **Azure-portaal** (https://portal.azure.com)



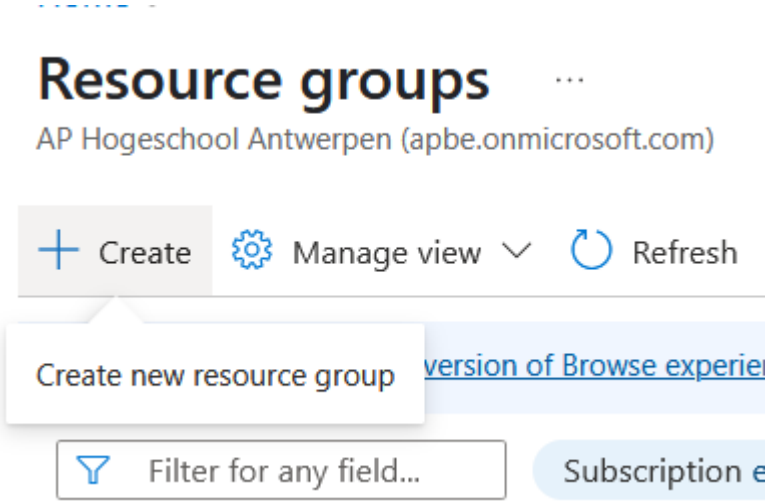
Stap 6:

Zoek naar **"Resource groups"** via de zoekbalk bovenaan en klik erop.



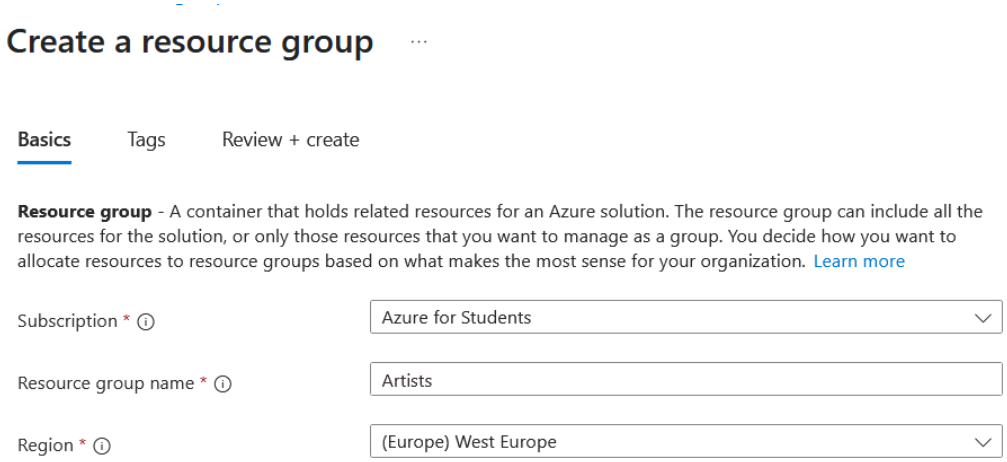
Stap 7:

Drukken op **“Create”**

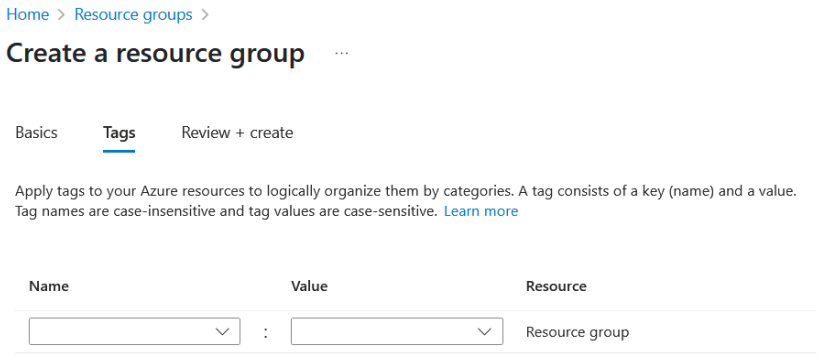


Stap 8:

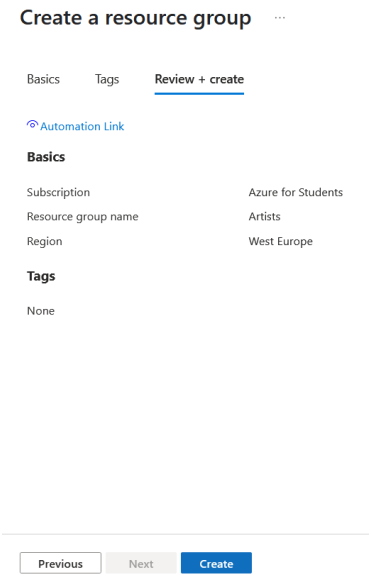
We kiezen een naam uit, voor onze sites zullen we gaan met **“Artists”**



Tags zijn niet nodig

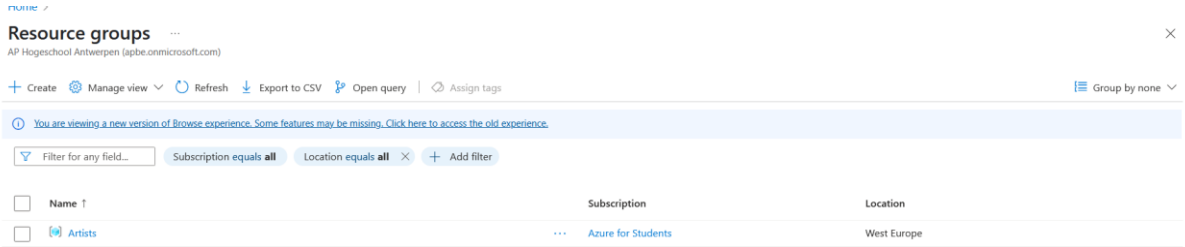


En dan gewoon **“Create”**



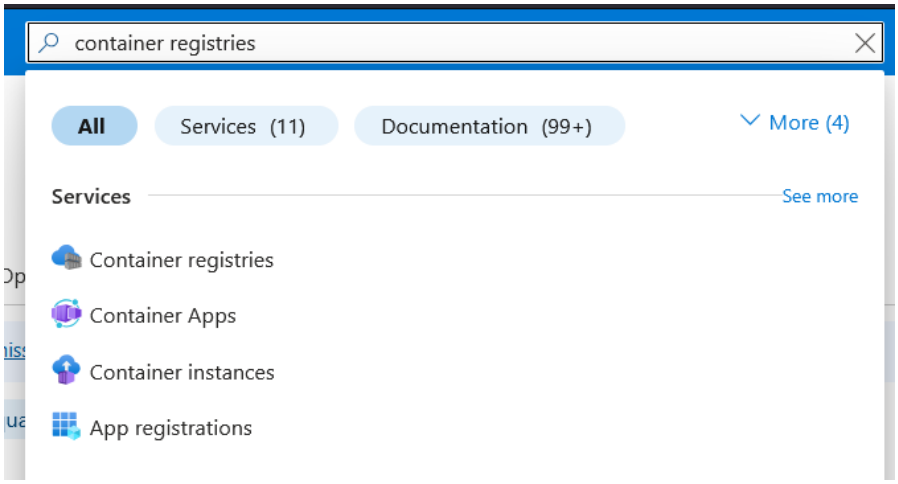
Stap 9:

Je ziet hier een overzicht van bestaande Resource Groups. In dit geval hebben we een groep genaamd rg-Artist, gekoppeld aan de Azure for Students-subscriptie en gehost in West Europe. Hierin zul je later je virtuele machine en andere resources aanmaken of beheren.



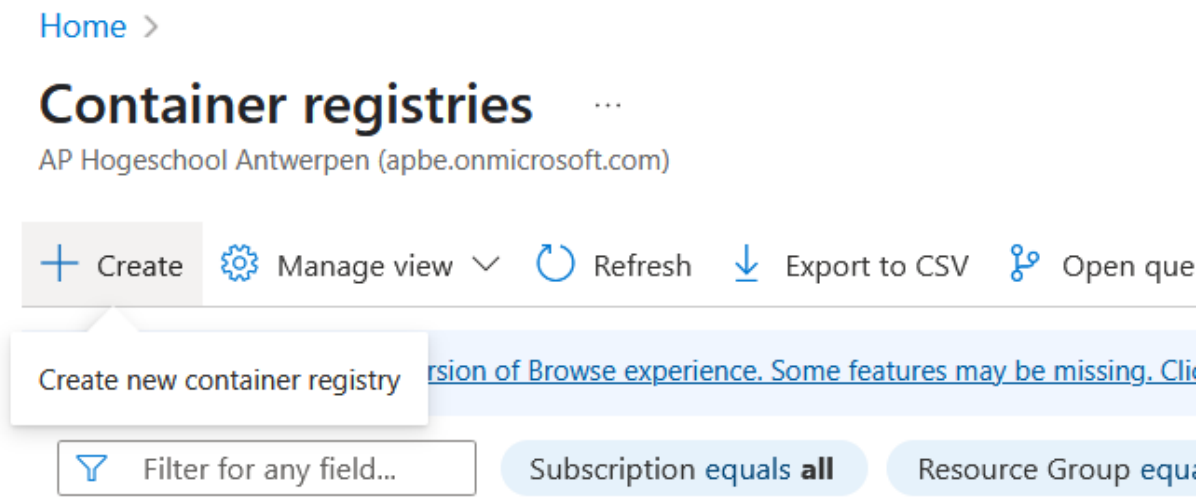
Stap 10:

Zoek naar **"Container registries"** via de zoekbalk bovenaan en klik erop.



Stap 11:

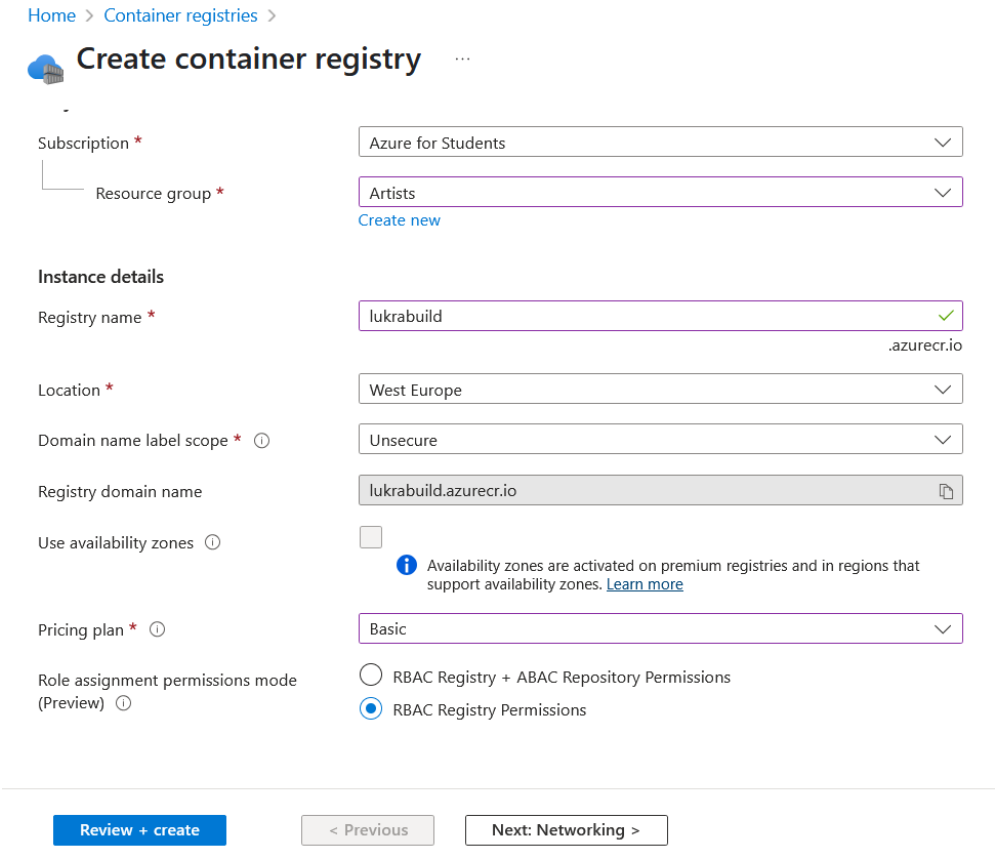
Druk op **“Create”**

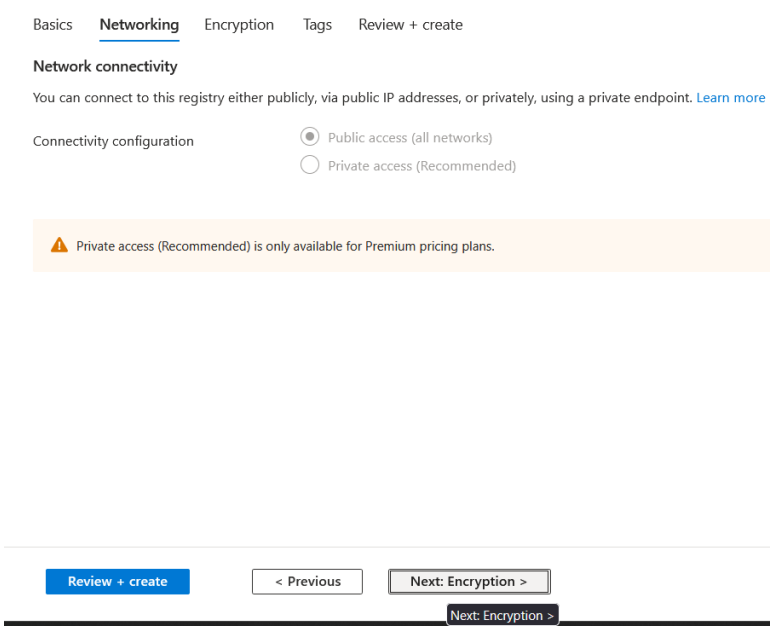


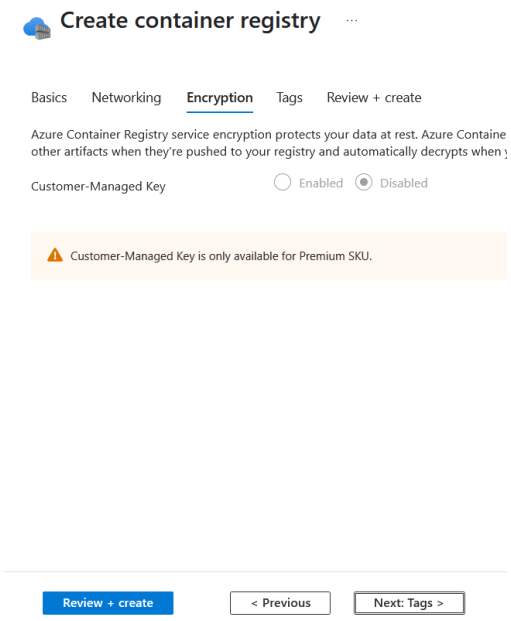
Stap 12:

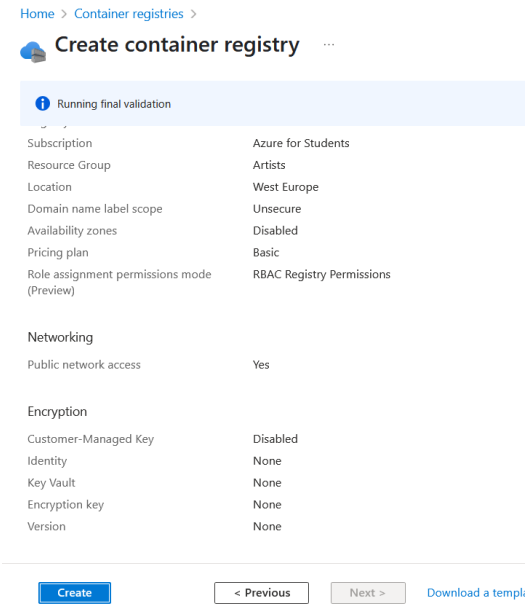
Voor de **“Resource group”** zullen we onze eerdere aangemaakte container kunnen vinden.

Naam is zelf te kiezen, ik zal in dit geval voor **“lukrabuild”** gaan (lukra is mijn gamer-tag)





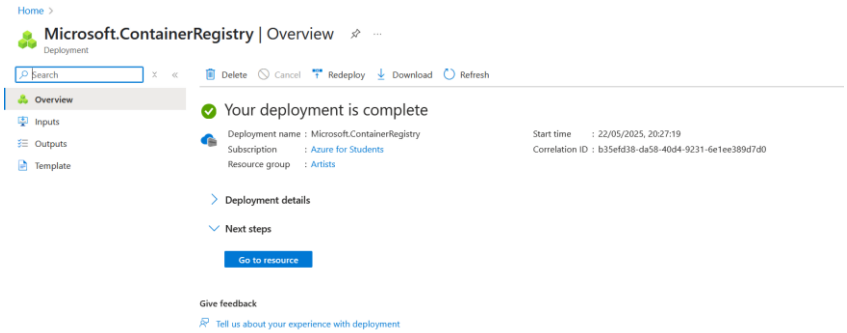


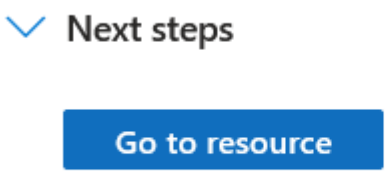


Er zijn geen settings nodig om aan te passen, gewoon verder gaan en creëren.

Stap 13:

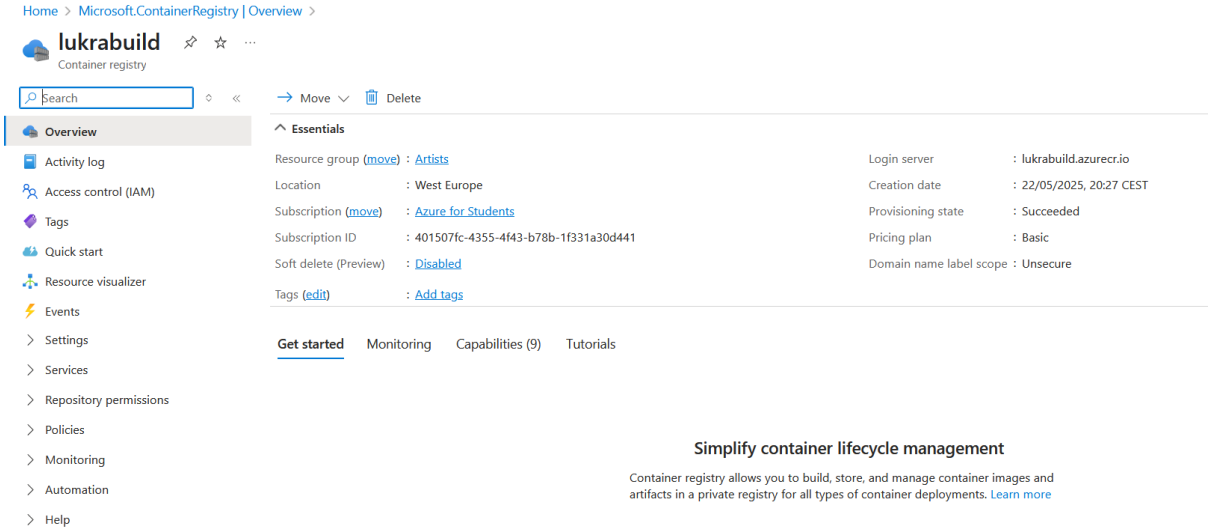
In deze stap zie je de bevestiging dat je Microsoft Container Registry correct is aangemaakt. We kunnen verder door op **“Go to resource”** te drukken.



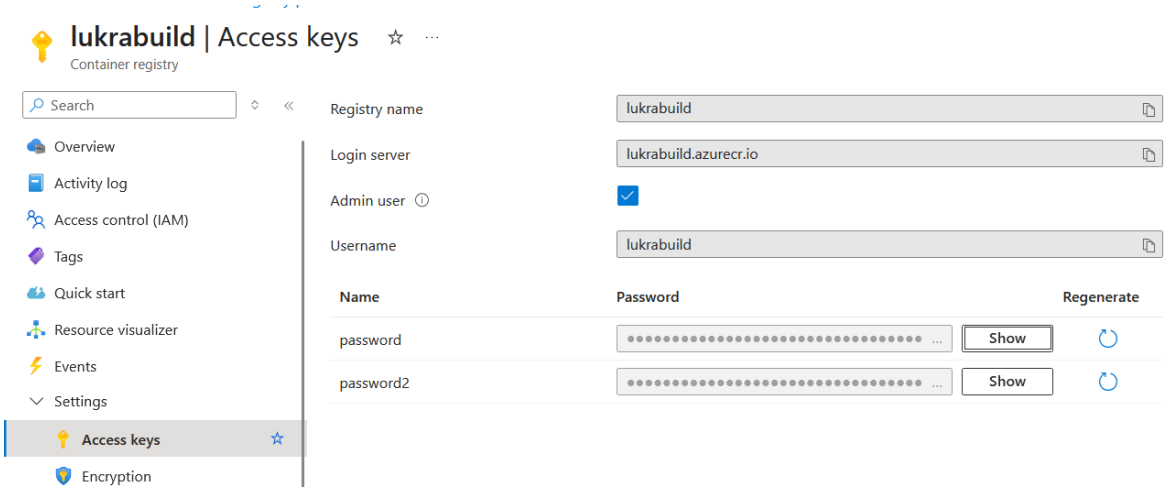


Stap 14:

Hier zie je een overzicht van onze container, we moeten nog iets kleins aanpassen in de **“Settings < Acces keys”**



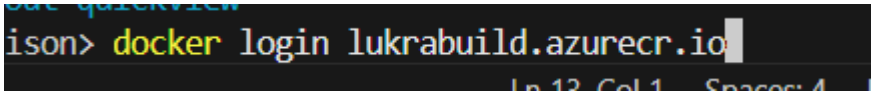
Admin user is unvinked, zeker vinken!

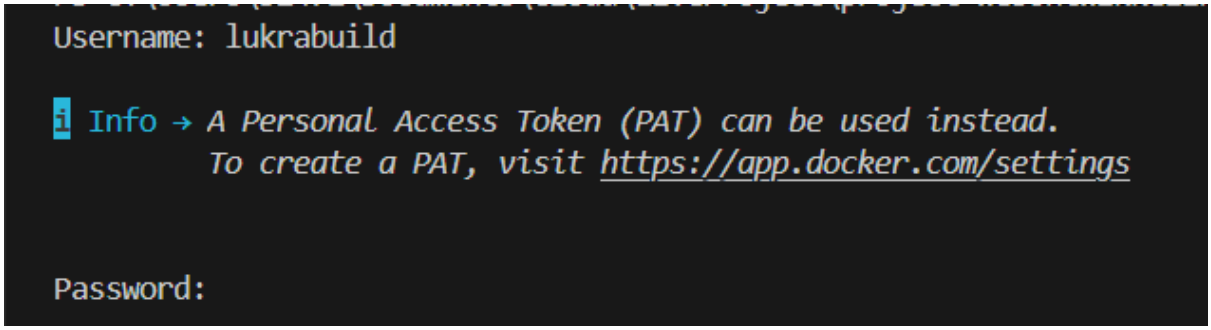


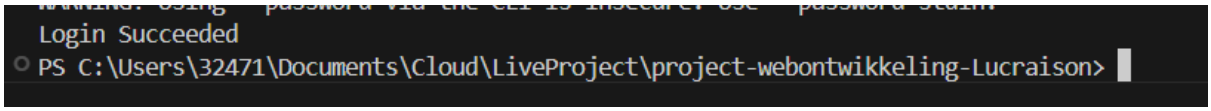
Stap 15:

We gaan nu verbinding maken met onze container, dit doen we met het commando:
docker login lukrabuild.azurecr.io

Het zal onze de login gegevens vragen, die vind je terug op Azure (stap 14)







Stap 16:

Zodra je succesvol bent ingelogd in Azure via de terminal, gebruik je de volgende commando's om je container te taggen en vervolgens te pushen naar je Azure Container Registry.
Zorg ervoor dat je je terminal opent in de juiste map waarin je het project hebt opgebouwd.

dokker tag artists-site lukrabuild.azurecr.io/artists-site

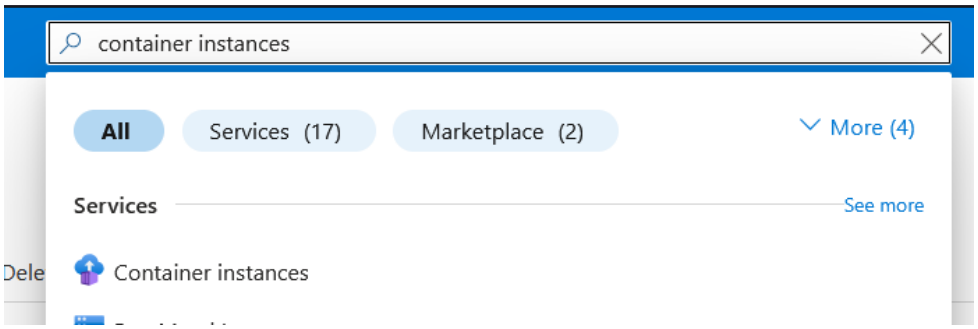
docker push lukrabuild.azurecr.io/artists-site

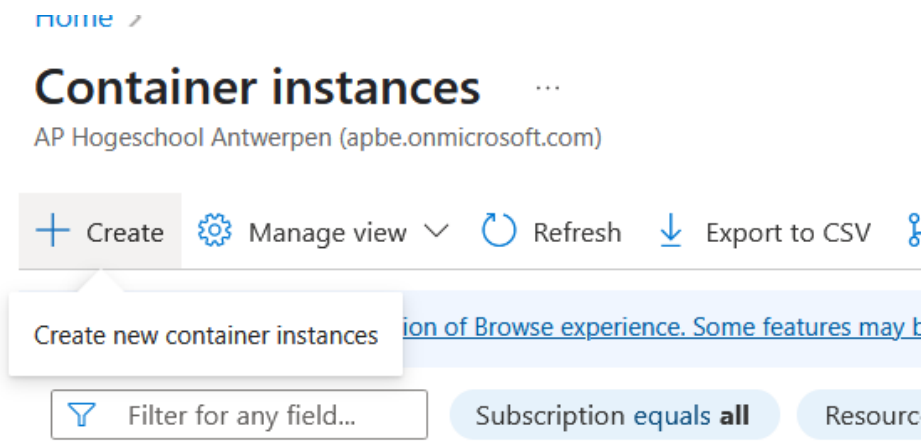
- Het eerste commando geeft jouw lokaal gebouwde container een nieuwe tag zodat Azure weet waar hij deze moet opslaan.
- Het tweede commando stuurt (pusht) de container naar je **Azure Container Registry** (lukrabuild.azurecr.io).

```
Login Succeeded
PS C:\Users\32471\Documents\Cloud\LiveProject\project-webontwikkeling-Lucraison> docker tag artists-site lukrabuild.azurecr.io/artists-site
PS C:\Users\32471\Documents\Cloud\LiveProject\project-webontwikkeling-Lucraison> docker push lukrabuild.azurecr.io/artists-site
Using default tag: latest
The push refers to repository [lukrabuild.azurecr.io/artists-site]
1644b64c7203: Pushed
398216956f98: Pushed
1391cdbaa87c: Pushed
e1e1b89fd646: Pushed
07bb3aacbadc: Pushed
82140d9a70a7: Pushed
f3b40b0cdb1c: Pushed
0b1f26057bd0: Pushed
08000c18d16d: Pushed
latest: digest: sha256:5a96b2c8a90311f2ca1bcdb6197c77bbdd1182e96eb425ce732c6e2d33c2742c size: 2205
PS C:\Users\32471\Documents\Cloud\LiveProject\project-webontwikkeling-Lucraison>
```

Stap 17:

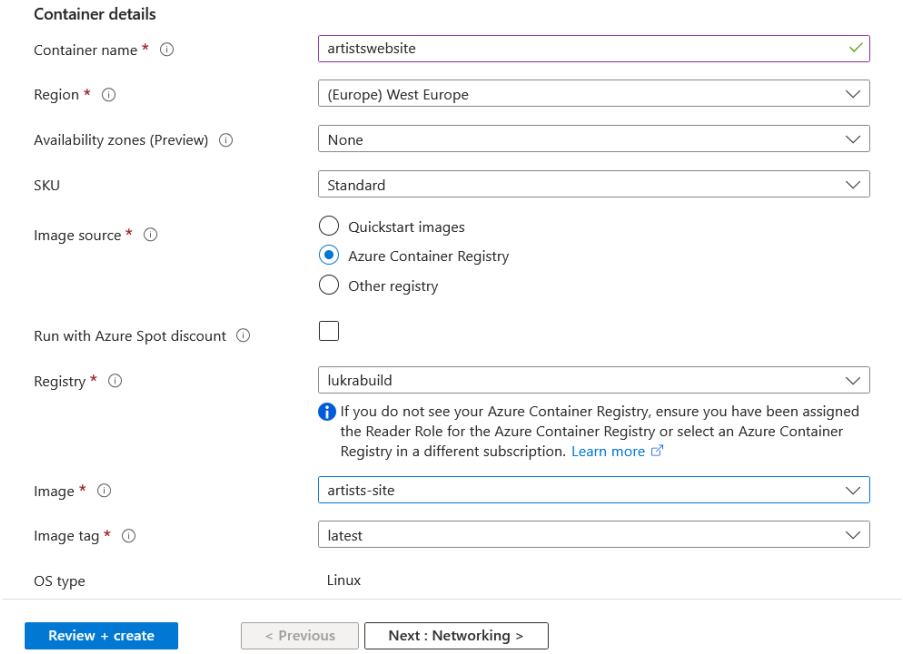
We gaan terug naar Azure en in de zoekbalk zoek je **"Container instances"** en je maakt er een nieuwe aan.



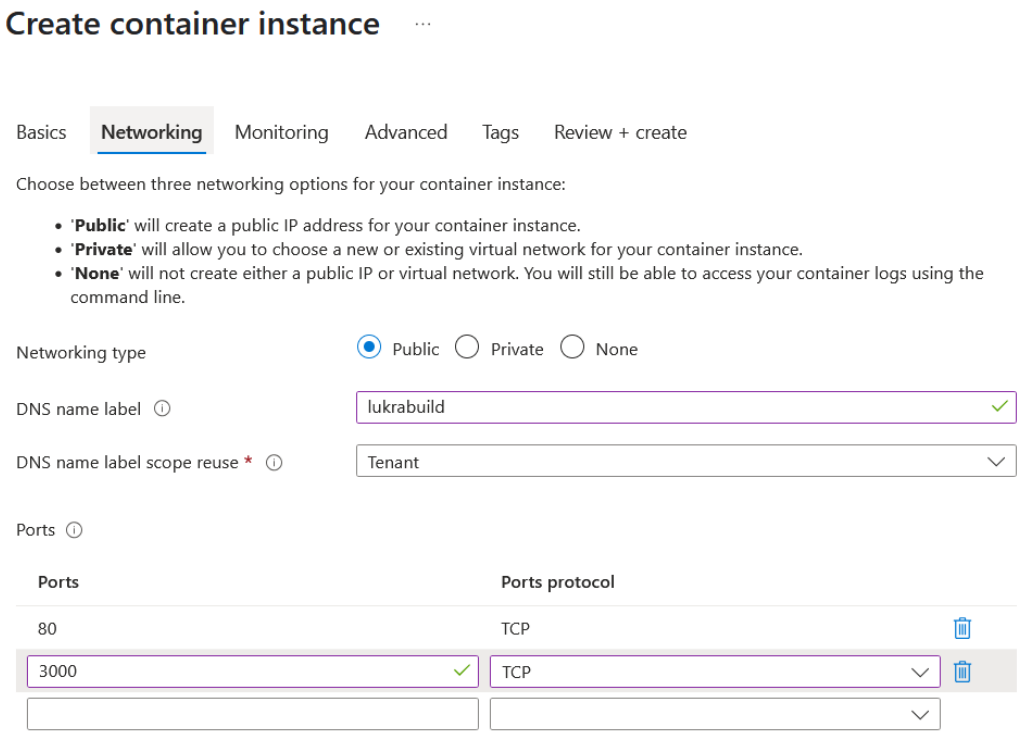


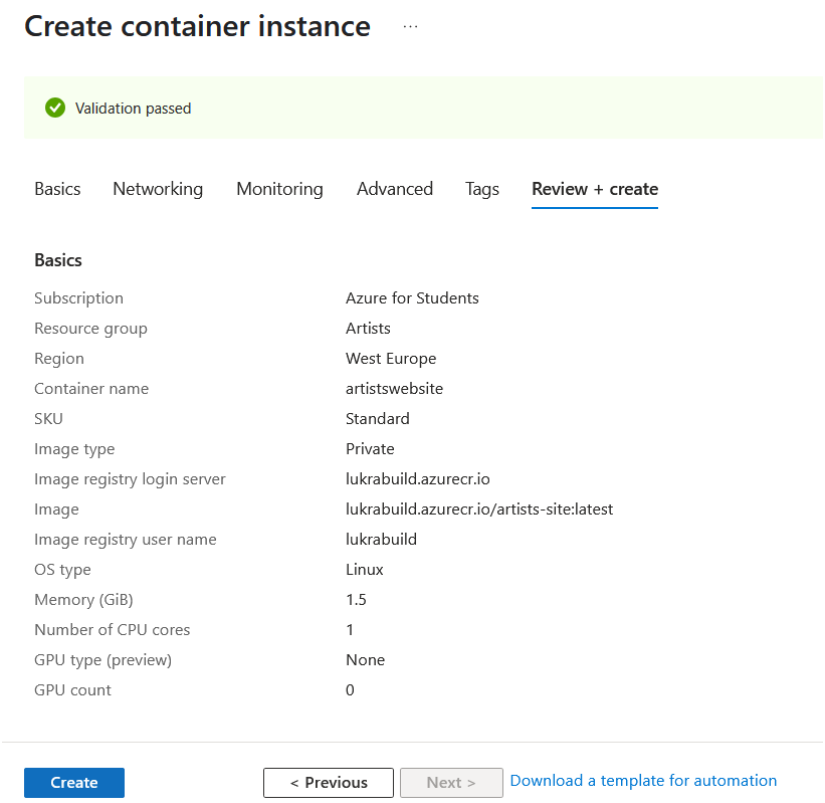
Container naam kiezen we voor **“artistswebsite”**

Image source: **Azure Container Registry**



Opmerking: Tijdens het aanmaken van de container instance heb ik ook poort 3000 toegevoegd uit gewoonte, omdat dit vaak gebruikt wordt bij lokale Node.js-apps. Voor deze productieomgeving bleek dit echter overbodig, aangezien de applicatie enkel via poort 80 bereikbaar is.

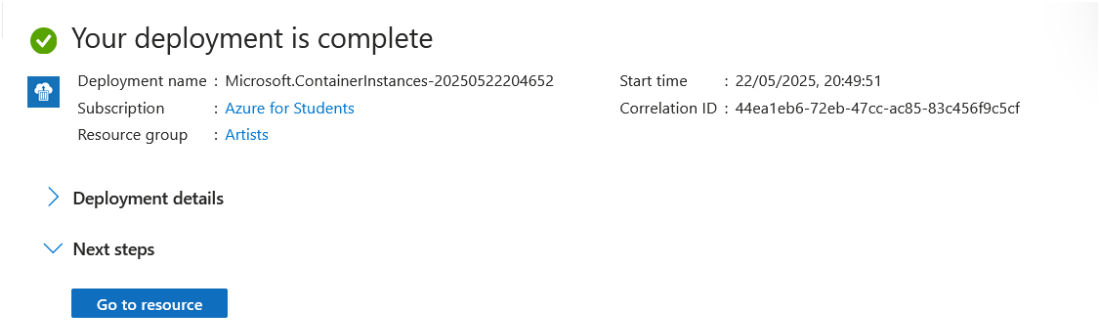


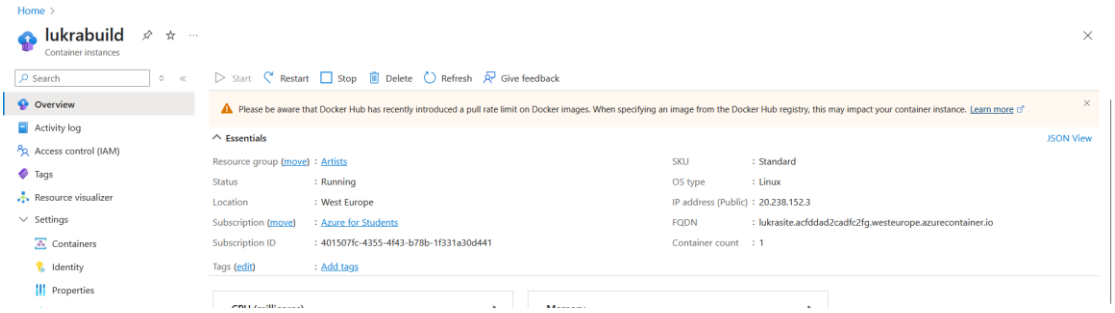


En dan **“Create”**

We zullen weer even moeten wachten tot de deployment compleet is

Maar nadat het klaar is **“Go to resource”**





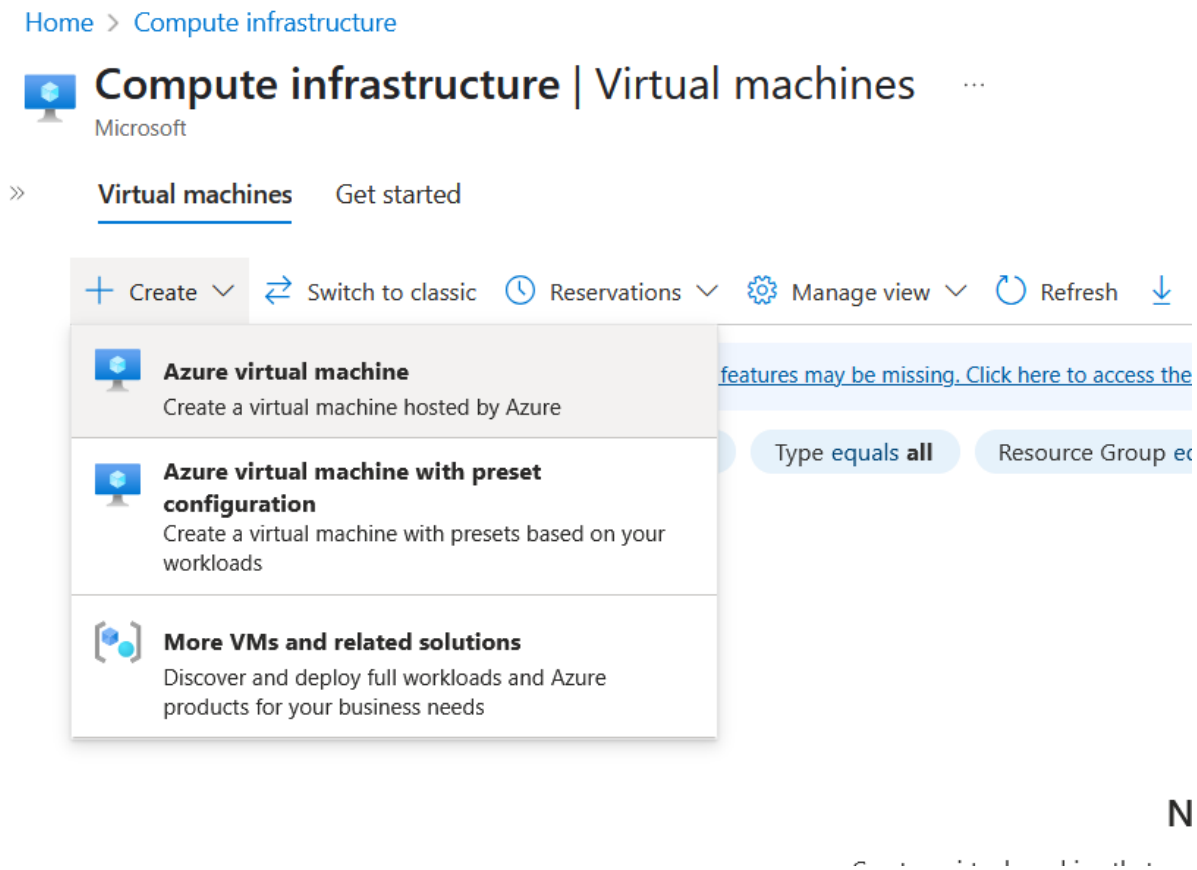
En het programma werkt en is te vinden op de link van FQDN:

Lukrasite.acfddad2cadf2fg.westeuropa.azurecontainer.io

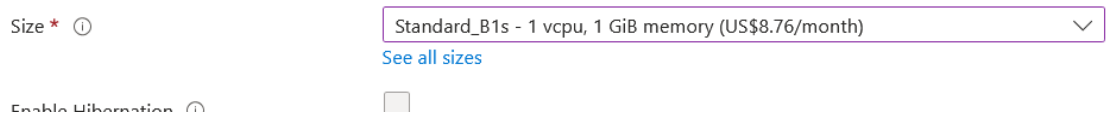
Stap 18:

We gaan ons project nu hosten via een Virtual Machine
Ga terug naar Azure en in de zoekbalk zoek je “Virtual machines”

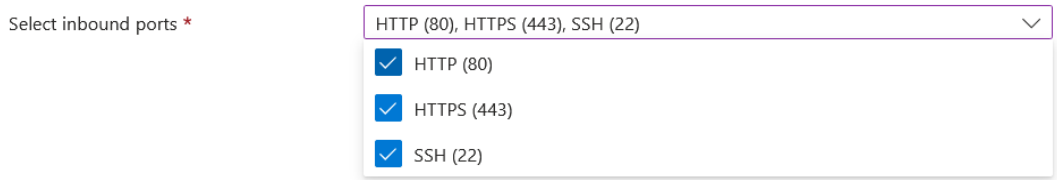
Op **“Create”** kies je voor **“Azure virtual machine”**



Bij de size druk op je op **“See all sizes”** en kies je voor de goedkoopst mogelijke (aangezien we op een budget werken) meer hebben we toch niet nodig.



En in ports:



- **HTTP (80):** Voor onversleutelde webverkeer naar je site.
- **HTTPS (443):** Voor versleuteld verkeer (Traefik + Let's Encrypt straks).
- **SSH (22):** Voor beheer via terminal (met je .pem key).

Basics | **Disks** | Networking | Management | Monitoring | Advanced | Tags | Review + create

Azure VMs have one operating system disk and a temporary disk for short-term storage. You can attach additional data disks. The size of the VM determines the type of storage you can use and the number of data disks allowed. Learn more

VM disk encryption

Azure disk storage encryption automatically encrypts your data stored on Azure managed disks (OS and data disks) at rest by default when persisting it to the cloud.

Encryption at host

Encryption at host is not registered for the selected subscription. Learn more

OS disk

OS disk size	Image default (30 GiB)
OS disk type *	Premium SSD (locally-redundant storage)
Delete with VM	☑
Key management	Platform-managed key
Enable Ultra Disk compatibility	☐

< Previous | Next : Networking > | Review + create

Validation passed

Help me create a low cost VM | Help me create a VM optimized for high availability | Help me choose the right VM size for my workload

Enable hotpatch	Off
Patch orchestration options	Image Default

Monitoring

Alerts	Off
Boot diagnostics	On
Enable OS guest diagnostics	Off
Enable application health monitoring	Off

Advanced

Extensions	None
VM applications	None
Cloud init	No
User data	No
Disk controller type	SCSI
Proximity placement group	None
Capacity reservation group	None

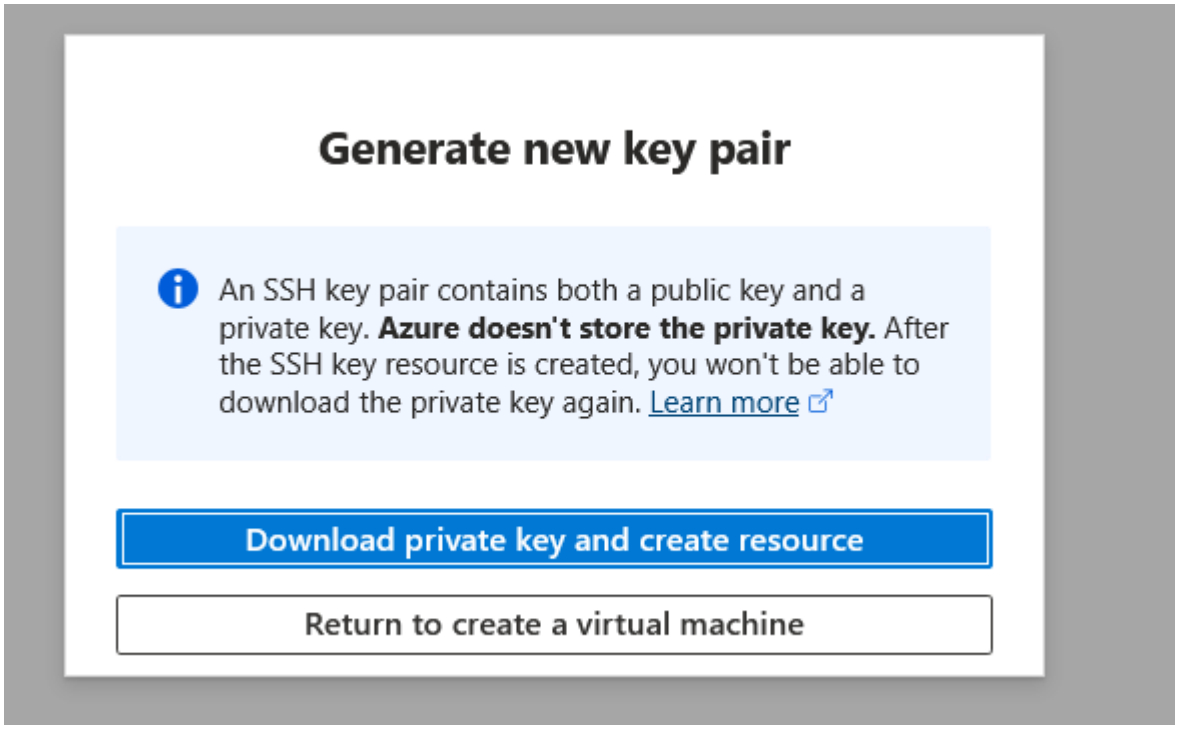
< Previous | Next > | Create

Verder geen opties aan te passen, enkel nog maar op **“Create”** drukken

Stap 19:

We zullen een prompt krijgen voor key pair om te verbinden met onze VM, deze sla je lokaal op

“Download private key and create resource”



We slaan dit op in onze map, met volgende pad:

C:\Users\32471\.ssh

En dan openen we een command prompt en typen het volgende commando om te verbinden met onze VM:

ssh -i C:\Users\32471\.ssh\lukramachine_key.pem azureuser@20.82.97.163

```
C:\Users\32471>ssh -i ~/.ssh/lukramachine_key.pem azureuser@20.82.97.163
```

We voegen -i toe met het pad van onze key zodat we toegang krijgen met onze privesleutel

Stap 20:

```
The authenticity of host '20.82.97.163 (20.82.97.163)' can't be established.
ED25519 key fingerprint is SHA256:+qY+mkhbG0/Cw2moA6c1eGoCHpYjuhzFvSn7iMjcT7o.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.82.97.163' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.11.0-1015-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat May 24 12:48:21 UTC 2025

  System load:  0.18               Processes:             114
  Usage of /:   5.5% of 28.02GB    Users logged in:       0
  Memory usage: 28%                IPv4 address for eth0: 10.0.0.4
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status



The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@lukramachine:~$ |
```

Eens hier volgen we met commandos

Installeer docker:

- **sudo apt update**
- **sudo apt install -y docker.io**

Start en activeer docker:

- **sudo systemctl start docker**
- **sudo systemctl enable docker**

(Optioneel) jezelf toevoegen om geen sudo te moeten doen:

- **sudo usermod -aG docker $USER**

Omdat we de laatste versie van ubuntu gebruiken heeft deze nog geen officiele package voor Docker Compose maar dit is makkelijk te regel met de volgende commando's:

- **mkdir -p ~/.docker/cli-plugins**
- **curl -SL https://github.com/docker/compose/releases/download/v2.24.4/docker-compose-linux-x86_64 -o ~/.docker/cli-plugins/docker-compose**
- **chmod +x ~/.docker/cli-plugins/docker-compose**

En tenslotte om te zien of alles juist werkt:

- **docker --version**
- **docker compose version**

```
azureuser@lukramachine:~$ docker --version
Docker version 26.1.3, build 26.1.3-0ubuntu1~24.04.1
```

```
azureuser@lukramachine:~$ docker compose version
Docker Compose version v2.24.4
```

En nu om het project op de VM te zetten zijn er enkele opties, ik kies ervoor om gebruik te make van het SCP commando, als men ervoor kiest om deze ook te gaan moet ik je waarschuwen

SCP (Secure Copy) zal een copy maken van onze lokale map naar de VM, deze kopie kan enorm zijn als je je node_modules map niet verwijdert! Zeker hiermee opletten.

Het commando dat ik uitvoer (op een normale command prompt trouwens) zal als volgt gaan:

scp -i C:/Users/32471/.ssh/lukramachine_key.pem -r "C:/Users/32471/Documents/Cloud/LiveProject/project-webontwikkeling-Lucraison" azureuser@20.82.97.163:/home/azureuser

Eens complete kunnen we zien in onze VM door het commando 'ls' uit te voeren dat onze map successvol is overgebracht.

```
azureuser@lukramachine:~$ ls
project-webontwikkeling-Lucraison
azureuser@lukramachine:~$ cd project-webontwikkeling-Lucraison/
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$ ls
Dockerfile  console-app.ts  dist           middleware        package.json  routes     session.ts     views
README.md   database.ts     interfaces.ts  package-lock.json  public        server.ts  tsconfig.json
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$
```

We moeten nog een docker-compose.yml aanmaken dit doen we met de volgende commandos:

- **touch docker-compose.yml**

En om deze aan te passen

- **nano docker-compose.yml**

We typen dit in Nano en slaan het op met CTRL + X, gevolgd door Y en Enter. (En nee, ik heb hier geen vijf minuten mee zitten sukkelen..)

```
  GNU nano 7.2
version: '3'
services:
  web:
    build: .
    ports:
      - "80:80"
    volumes:
      - .:/app
    restart: unless-stopped
```

De dockerfile moet nog aangepast worden

```
FROM node:18-alpine

WORKDIR /app

# Kopieer package.json en package-lock.json
COPY package*.json ./

# Installeer alle dependencies, inclusief devDependencies (zoals TypeScript)
RUN npm install

# Kopieer de rest van je projectbestanden
COPY . .

# Build de TypeScript-code
RUN npx tsc

# Open poort 80
EXPOSE 80
ENV PORT=80

# Start de app
CMD ["node", "dist/server.js"]
```

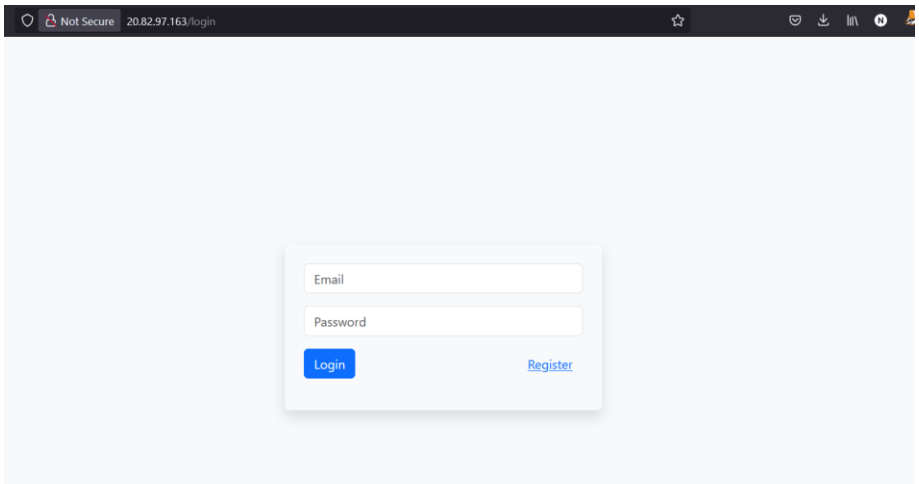
Nog snel dan de docker commandos om het te laten werken

Docker compose build

Docker compose up -d

```
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$ docker compose build
[+] Building 78.8s (11/11) FINISHED                                                    docker:default
 => [web internal] load build definition from Dockerfile                                          0.1s
 => => transferring dockerfile: 434B                                                              0.0s
 => [web internal] load metadata for docker.io/library/node:18-alpine                             0.8s
 => [web internal] load .dockerignore                                                             0.0s
 => => transferring context: 2B                                                                   0.0s
 => [web 1/6] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc  0.0s
 => [web internal] load build context                                                             8.6s
 => => transferring context: 92.06MB                                                              8.4s
 => CACHED [web 2/6] WORKDIR /app                                                                 0.0s
 => [web 3/6] COPY package*.json ./                                                               0.7s
 => [web 4/6] RUN npm install                                                                    15.3s
 => [web 5/6] COPY . .                                                                           18.8s
 => [web 6/6] RUN npx tsc                                                                        10.6s
 => [web] exporting to image                                                                     21.5s
 => => exporting layers                                                                          21.1s
 => => writing image sha256:7f68fd057cfe20561512297b73e9110af27be77f27c03eaf418b3441f9b316bf       0.1s
 => => naming to docker.io/library/project-webontwikkeling-lucraison-web                          0.0s
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$ docker-compose up -d
WARN[0001] /home/azureuser/project-webontwikkeling-Lucraison/docker-compose.yml: the attribute `version` is obsolete, it
 will be ignored, please remove it to avoid potential confusion
[+] Running 2/2
 ✔Network project-webontwikkeling-lucraison_default  Creat...                                    0.5s
 ✔Container project-webontwikkeling-lucraison-web-1  Start...                                    4.1s
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$
```

En voila, de site val zicht te vinden op de IP van de VM (die vind je terug op Azure maar we gaan direct nog verder voor de DNS): 20.82.97.163



Stap 21:

Voor traefik

Maak een map aan in je project met de naam **‘traefik’**

```
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$ mkdir traefik
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$ ls
Dockerfile      database.ts         interfaces.ts  package-lock.json  routes      traefik
README.md       dist                middleware     package.json       server.ts   tsconfig.json
```

Daar voegen we 2 bestanden aan toe

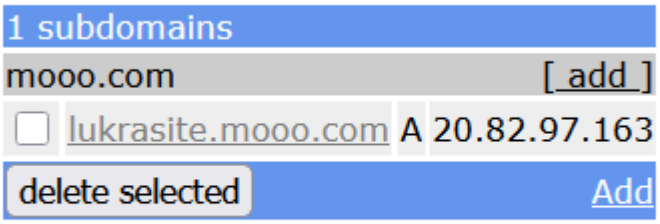
traefik.yml en **acme.json**

```
  GNU nano 7.2                                                    traefik.yml
entryPoints:
  web:
    address: ":80"
  websecure:
    address: ":443"

providers:
  docker:
    exposedByDefault: false

certificatesResolvers:
  letsencrypt:
    acme:
      email: s117195@ap.be
      storage: /letsencrypt/acme.json
      httpChallenge:
        entryPoint: web
```

sudo chmod 600 traefik/acme.json

we moeten ook een DNS aanmaken dit doen we op

https://freedns.afraid.org/



We hebben ons domein gemaakt, dit moet type A zijn en voor de IP voeren we dat van de VM in natuurlijk

Nog even de docker compose aanpassen

```yaml
version: "3.8"

services:
  traefik:
    image: traefik:v2.11
    command:
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.websecure.address=:443"
      - "--certificatesresolvers.letsencrypt.acme.httpchallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.httpchallenge.entrypoint=web"
      - "--certificatesresolvers.letsencrypt.acme.email=s117195@ap.be"
      - "--certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json"
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - "./traefik/traefik.yml:/etc/traefik/traefik.yml"
      - "./traefik/acme.json:/letsencrypt/acme.json"
      - "/var/run/docker.sock:/var/run/docker.sock"

  web:
    build: .
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.web.rule=Host(`lukrasite.mooo.com`)"
      - "traefik.http.routers.web.entrypoints=web"
      - "traefik.http.routers.websecure.rule=Host(`lukrasite.mooo.com`)"
      - "traefik.http.routers.websecure.entrypoints=websecure"
      - "traefik.http.routers.websecure.tls.certresolver=letsencrypt"
```

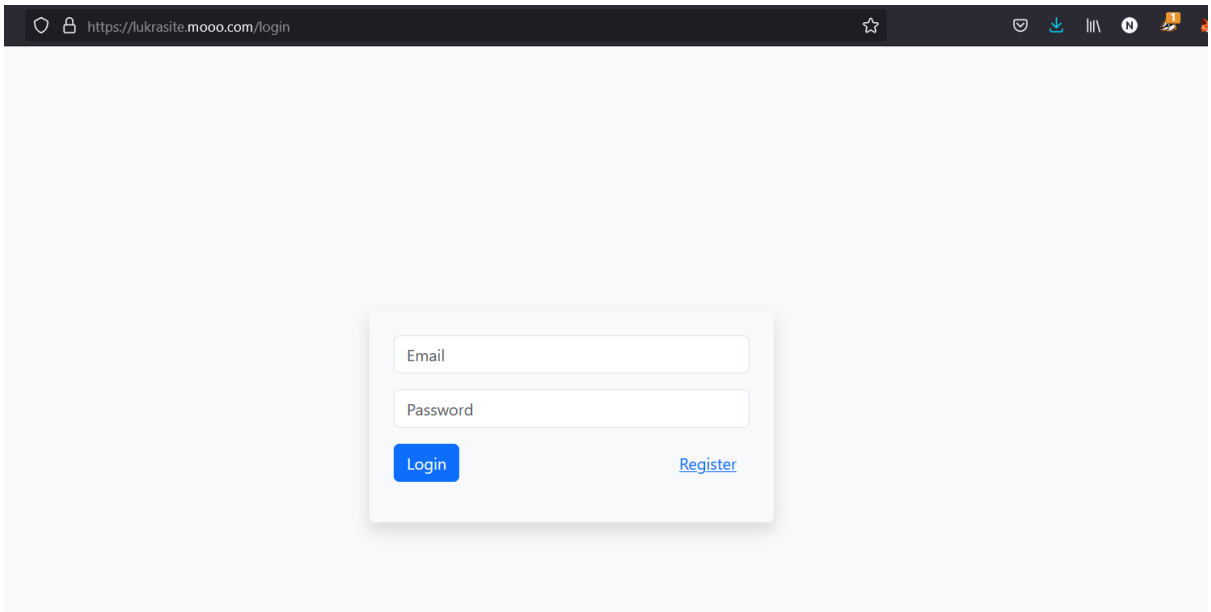
We voegen hier de link toe van onze gratis DNS

docker compose down

docker compose up -d

```
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$ docker compose up -d
[+] Running 2/2
 ✔Container project-webontwikkeling-lucraison-web-1      S...                    0.0s
 ✔Container project-webontwikkeling-lucraison-traefik-1  Started                 0.0s
azureuser@lukramachine:~/project-webontwikkeling-Lucraison$
```

En onze webpagina werkt



https://lukrasite.mooo.com/

Reflectie

Tijdens het project liep ik tegen een probleem aan waarbij mijn TypeScript-project niet werkte in de Docker-container op Azure. Eerst dacht ik dat ts-node wel zou volstaan, maar dat bleek niet geschikt voor productie. Na wat troubleshootwerk ontdekte ik dat ik mijn TypeScript moest omzetten naar JavaScript via tsc. Door die stap toe te voegen, werkte alles zoals het hoort. Ik heb hieruit geleerd dat een project klaarmaken voor productie net iets meer vraagt dan gewoon lokaal draaien.

Voor de rest ben ik weinig tot geen grote problemen tegengekomen. Dankzij de enorme hoeveelheid aan informatie online tegenwoordig én de hulp van AI-tools die enorm goed zijn in het analyseren en debuggen van fouten, kon ik vaak snel tot een oplossing komen. Dat heeft me heel wat tijd bespaard en hielp me vooral bij de meer technische struikelblokken.